

UNIVERSITY OF **Baguio**

SCHOOL OF INFORMATION TECHNOLOGY



WEBSYS1

Web Systems and Technologies 1

UNIT 4: PHP Data Objects (PDO)

Topic Overview

Modern PHP applications frequently communicate with relational databases to store, retrieve, modify, and delete information. PHP Data Objects, commonly known as PDO, provides a structured and object-oriented interface for communicating with databases.

PDO allows PHP applications to:

- establish database connections;
- execute SQL statements;
- retrieve database records;
- insert new records;
- update existing records;
- delete records;
- use prepared statements;
- bind values securely;
- handle database errors;
- retrieve query results;
- protect applications against SQL injection.

In this module, students will progress from a basic PDO connection to a complete PHP OOP + PDO CRUD application.

Learning Outcomes

At the end of the lesson, the student should be able to:

1. Define PHP Data Objects (PDO).
2. Explain the purpose of PDO in PHP applications.
3. Identify the major components of a PDO database connection.
4. Establish a database connection using PDO.
5. Configure PDO error handling.
6. Create and execute SQL statements using PDO.
7. Explain the purpose of prepared statements.
8. Use `prepare()` and `execute()`.
9. Use `bindParam()` and `bindValue()`.
10. Retrieve database records using `fetch()` and `fetchAll()`.
11. Implement INSERT operations using PDO.
12. Implement SELECT operations using PDO.
13. Implement UPDATE operations using PDO.
14. Implement DELETE operations using PDO.
15. Implement database searching using PDO.
16. Validate user input before database processing.
17. Handle database errors appropriately.

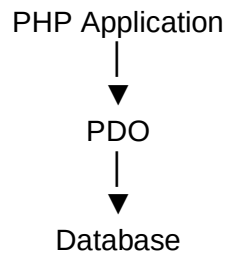
18. Explain how prepared statements help prevent SQL injection.
19. Integrate PDO with PHP classes and objects.
20. Develop a complete CRUD application using PHP OOP and PDO.

What is PDO

PDO stands for: PHP Data Objects

PDO is a PHP extension that provides an interface for communicating with databases. Instead of writing database-specific PHP code for every database system, PDO provides a common interface.

For example:



PDO can communicate with several database systems, including:

- MySQL
- MariaDB
- PostgreSQL
- SQLite
- Microsoft SQL Server
- Oracle

For this module, students will use:

PHP + PDO + MySQL/MariaDB

Why Use PDO?

A PHP application needs a way to communicate with a database. PDO provides:

1. Object-oriented database interaction
 - PDO uses objects and methods.Example:
`$pdo = new PDO(...);`
2. Prepared statements
 - PDO supports:
`$stmt = $pdo->prepare($sql);`
Prepared statements help protect applications against SQL injection.
3. Error handling
 - PDO can throw exceptions when database errors occur.
4. Consistent interface
 - The same general PDO concepts can be used with different database systems.

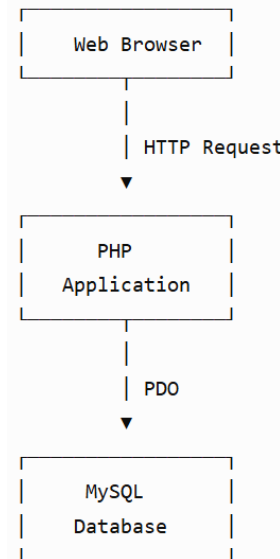
PDO VS MYSQLI

Feature	PDO	MySQLi
Object-oriented programming	Yes	Yes
Prepared statements	Yes	Yes
MySQL support	Yes	Yes
Multiple database systems	Yes	No

Exception-based error handling	Yes	Yes
Recommended for this module	Yes	No

BASIC DATABASE ARCHITECTURE

A PHP application communicates with a database through PDO.



The browser does not directly communicate with MySQL.

Instead: *Browser => PHP => PDO => MySQL*

Example: Database Setup

Create a database:

```

CREATE DATABASE student_system;
Select the database:
USE student_system;
Create the students table:
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id VARCHAR(20) NOT NULL,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL,
  course VARCHAR(100) NOT NULL
);
Insert sample records:
INSERT INTO students
(student_id, name, email, course)
VALUES
('2026-001', 'Juan Dela Cruz', 'juan@example.com', 'BSIT'),
('2026-002', 'Maria Santos', 'maria@example.com', 'BSCS'),
('2026-003', 'Pedro Reyes', 'pedro@example.com', 'BSIT');
  
```

PDO Connection

The first task is establishing a database connection.

Basic syntax:

```
$pdo = new PDO(
    "mysql:host=localhost;dbname=student_system",
    "root",
    ""
);
```

The three important components are:

- Database Server
- Database Name
- Username and Password

FIRST PDO PROGRAM

Create: **index.php**

```
<?php
$pdo = new PDO(
    "mysql:host=localhost;dbname=student_system",
    "root",
    ""
);
echo "Database connection successful.";
?>
```

Expected output: Database connection successful.

WHY ERROR HANDLING IS IMPORTANT

A database connection can fail.

Possible reasons include:

- MySQL is not running;
- incorrect database name;
- incorrect username;
- incorrect password;
- database server unavailable.

Therefore, the application should handle errors.

PDO Exception Handling

```
<?php
try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=student_system",
        "root",
        ""
    );
    echo "Database connection successful.";
}
catch(PDOException $e) {
    echo "Database connection failed.";
}
?>
```

PDO::ATTR_ERRMODE

```
<?php
```

```
try {
    $pdo = new PDO(
        "mysql:host=localhost;dbname=student_system",
        "root",
        ""
    );
    $pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );
    echo "Connected successfully.";
}
catch(PDOException $e) {
    echo "Connection failed.";
}
?>
```

PDO CONNECTION FILE

Instead of creating the connection repeatedly, create a separate file.

Project:

```
student_system/
|
|— config/
|   |— Database.php
|
|— index.php
```

Database.php

```
<?php
class Database
{
    private $host = "localhost";
    private $database = "student_system";
    private $username = "root";
    private $password = "";
    public function connect()
    {
        try {
            $pdo = new PDO(
                "mysql:host={$this->host};dbname={$this->database}",
                $this->username,
                $this->password
            );
            $pdo->setAttribute(
                PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION
            );
        }
    }
}
```

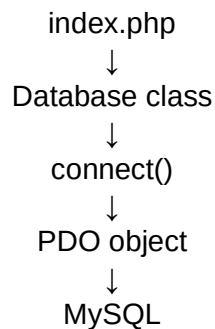
```
        return $pdo;
    }
    catch(PDOException $e) {
        die("Database connection failed.");
    }
}
}
```

USING THE DATABASE CLASS

index.php

```
<?php
require_once "config/Database.php";
$databse = new Database();
$pdo = $databse->connect();
echo "Database connection successful.";
?>
```

The process is:



EXECUTING SQL WITH PDO

PDO can execute SQL statements. For simple SQL without user input:

```
$sql = "SELECT * FROM students";
$result = $pdo->query($sql);
```

However, when user input is involved, students should use: Prepared statements

PDO QUERY()

```
$sql = "SELECT * FROM students";
$result = $pdo->query($sql);
```

query() executes an SQL statement directly. It can be useful for simple statements that contain no user-supplied values. However, it should not be used by simply concatenating user input into SQL.

PREPARED STATEMENTS

```
$sql = "SELECT * FROM students WHERE id = :id";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    "id" => 1
]);
```

WHY PREPARED STATEMENTS?

Consider this unsafe approach:

```
$id = $_GET["id"];
```

```
$sql = "SELECT * FROM students WHERE id = '$id'";  
$result = $pdo->query($sql);
```

The user controls \$id. This can create an SQL injection vulnerability.

Instead:

```
$sql = "SELECT * FROM students WHERE id = :id";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([  
    "id" => $id  
]);
```

The value is treated as data rather than SQL code.

PREPARE()

The prepare() method prepares an SQL statement.

Example:

```
$sql = "  
    SELECT *  
    FROM students  
    WHERE course = :course  
";  
  
$stmt = $pdo->prepare($sql);
```

The :course portion is a named parameter.

EXECUTE()

After preparing the statement:

```
$stmt->execute();
```

If parameters are required:

```
$stmt->execute([  
    "course" => "BSIT"  
]);
```

Example:

```
$sql = "  
    SELECT *  
    FROM students  
    WHERE course = :course  
";  
$stmt = $pdo->prepare($sql);  
$stmt->execute([  
    "course" => "BSIT"  
]);
```

NAMED PARAMETERS

Example: SELECT * FROM students WHERE id = :id

Here: :id is a named parameter.

PHP Example:

```
$stmt->execute([  
    "id" => $id  
]);
```

Another Example:

```
INSERT INTO students(student_id, name, email, course)  
VALUES (:student_id, :name, :email, :course)
```

bindParam()

PDO provides bindParam().

Example:

```
$id = 1;  
$sql = "  
    SELECT *  
    FROM students  
    WHERE id = :id  
";  
$stmt = $pdo->prepare($sql);  
$stmt->bindParam(":id", $id);  
$stmt->execute();
```

bindValue()

Another option is: \$stmt->bindValue(":id", \$id);

Example:

```
$id = 1;  
$stmt->bindValue(":id", $id);  
$stmt->execute();
```

bindParam() VS bindValue()

Feature	bindParam()	bindValue()
Binds a variable	Yes	Value
Variable evaluated at execution	Yes	No
Common beginner usage	Moderate	Easy
Suitable for fixed value	Less convenient	Yes

SELECT OPERATION

To retrieve records:

```
$sql = "SELECT * FROM students";  
$stmt = $pdo->prepare($sql);  
$stmt->execute();
```

The result can then be retrieved.

fetch() - fetch() retrieves one record.

```
$student = $stmt->fetch(PDO::FETCH_ASSOC);
```

Example:

```
$sql = "  
    SELECT *  
    FROM students
```

```
WHERE id = :id
";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    "id" => 1
]);
$student = $stmt->fetch(PDO::FETCH_ASSOC);
echo $student["name"];
```

PDO::FETCH_ASSOC

The constant PDO::FETCH_ASSOC returns the record as an associative array.

```
[
    "id" => 1,
    "student_id" => "2026-001",
    "name" => "Juan Dela Cruz",
    "email" => "juan@example.com",
    "course" => "BSIT"
]
```

Therefore: echo \$student["name"];
produces: Juan Dela Cruz

fetchAll() - fetchAll() retrieves multiple records.

```
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Example:

```
$sql = "SELECT * FROM students";
$stmt = $pdo->prepare($sql);
$stmt->execute();
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

DISPLAYING MULTIPLE RECORDS

```
foreach($students as $student)
{
    echo $student["student_id"];
    echo " - ";
    echo $student["name"];
    echo "<br>";
}
```

Output:

```
2026-001 - Juan Dela Cruz
2026-002 - Maria Santos
2026-003 - Pedro Reyes
```

SELECT WITH WHERE

```
$sql = "
    SELECT *
    FROM students
```

```
WHERE course = :course
";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    "course" => "BSIT"
]);
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

SELECT WITH LIKE

Searching can be performed using LIKE.

Example:

```
$sql = "
    SELECT *
    FROM students
    WHERE name LIKE :keyword
";

$keyword = "%Juan%";

$stmt = $pdo->prepare($sql);
$stmt->execute([
    "keyword" => $keyword
]);
```

INSERT OPERATION

To insert a record:

```
$sql = "
    INSERT INTO students
    (student_id, name, email, course)
    VALUES
    (:student_id, :name, :email, :course)
";

$stmt = $pdo->prepare($sql);

$stmt->execute([
    "student_id" => "2026-004",
    "name" => "Ana Cruz",
    "email" => "ana@example.com",
    "course" => "BSIT"
]);
```

COMPLETE INSERT EXAMPLE

```
<?php
require_once "config/Database.php";
$database = new Database();
$pdo = $database->connect();

$sql = "
```

```
INSERT INTO students
(student_id, name, email, course)
VALUES
(:student_id, :name, :email, :course)
";

$stmt = $pdo->prepare($sql);

$stmt->execute([
    "student_id" => "2026-004",
    "name" => "Ana Cruz",
    "email" => "ana@example.com",
    "course" => "BSIT"
]);
echo "Student successfully added.";
?>
```

UPDATE OPERATION

```
UPDATE students
SET
    name = :name,
    email = :email,
    course = :course
WHERE id = :id

$sql = "
    UPDATE students
    SET
        name = :name,
        email = :email,
        course = :course
    WHERE id = :id
";

$stmt = $pdo->prepare($sql);

$stmt->execute([
    "name" => "Juan Updated",
    "email" => "juanupdated@example.com",
    "course" => "BSCS",
    "id" => 1
]);
```

DELETE OPERATION

```
DELETE FROM students
WHERE id = :id

$sql = "
    DELETE FROM students
```

```
WHERE id = :id
";

$stmt = $pdo->prepare($sql);

$stmt->execute([
    "id" => 1
]);
```

ROWCOUNT()

PDO can report the number of affected rows.

```
$count = $stmt->rowCount();
```

Example:

```
$stmt->execute([
    "id" => 1
]);
if($stmt->rowCount() > 0)
{
    echo "Record deleted.";
}
else
{
    echo "No record was deleted.";
}
```

HTML FORM + PDO

Now connect PDO with an HTML form.

```
<form method="POST">

    <label>Student ID</label>
    <input type="text" name="student_id">

    <label>Name</label>
    <input type="text" name="name">

    <label>Email</label>
    <input type="email" name="email">

    <label>Course</label>
    <input type="text" name="course">

    <button type="submit">
        Save
    </button>

</form>
```

PROCESSING THE FORM

```
if($_SERVER["REQUEST_METHOD"] === "POST")
{
    $student_id = $_POST["student_id"];
    $name = $_POST["name"];
    $email = $_POST["email"];
    $course = $_POST["course"];
}
```

INPUT VALIDATION

Never immediately trust: \$_POST

Validate the input first.

Example:

```
$student_id = trim($_POST["student_id"] ?? "");
$name = trim($_POST["name"] ?? "");
$email = trim($_POST["email"] ?? "");
$course = trim($_POST["course"] ?? "");
```

Then:

```
if(
    $student_id === "" ||
    $name === "" ||
    $email === "" ||
    $course === ""
)
{
    echo "All fields are required.";
}
```

EMAIL VALIDATION

PHP provides: filter_var()

Example:

```
if(!filter_var($email, FILTER_VALIDATE_EMAIL))
{
    echo "Invalid email address.";
}
```

COMPLETE INSERT FORM

```
<?php
require_once "config/Database.php";
$database = new Database();
$pdo = $database->connect();
$message = "";

if($_SERVER["REQUEST_METHOD"] === "POST")
{
    $student_id = trim($_POST["student_id"] ?? "");
    $name = trim($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $course = trim($_POST["course"] ?? "");
```

```

if(
    $student_id === "" ||
    $name === "" ||
    $email === "" ||
    $course === ""
)
{
    $message = "All fields are required.";
}
elseif(!filter_var($email, FILTER_VALIDATE_EMAIL))
{
    $message = "Invalid email address.";
}
else
{
    $sql = "
        INSERT INTO students
        (student_id, name, email, course)
        VALUES
        (:student_id, :name, :email, :course)
    ";

    $stmt = $pdo->prepare($sql);

    $stmt->execute([
        "student_id" => $student_id,
        "name" => $name,
        "email" => $email,
        "course" => $course
    ]);

    $message = "Student successfully added.";
}
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Add Student</title>
</head>
<body>
<h1>Add Student</h1>
<p>
    <?= htmlspecialchars($message) ?>
</p>
<form method="POST">
    <label>Student ID</label>

```

```
<input type="text" name="student_id">

<br><br>

<label>Name</label>
<input type="text" name="name">

<br><br>

<label>Email</label>
<input type="email" name="email">

<br><br>

<label>Course</label>
<input type="text" name="course">

<br><br>

<button type="submit">
    Save Student
</button>
</form>
</body>
</html>
```

READING DATA INTO A TABLE

```
PHP:
$sql = "
    SELECT *
    FROM students
    ORDER BY id DESC
";
$stmt = $pdo->prepare($sql);
$stmt->execute();
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

```
HTML:
<table border="1">
<tr>
    <th>ID</th>
    <th>Student ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Course</th>

</tr>
```

```
<?php foreach($students as $student): ?>
```

```
<tr>

    <td>
        <?= htmlspecialchars($student["id"]) ?>
    </td>

    <td>
        <?= htmlspecialchars($student["student_id"]) ?>
    </td>

    <td>
        <?= htmlspecialchars($student["name"]) ?>
    </td>

    <td>
        <?= htmlspecialchars($student["email"]) ?>
    </td>

    <td>
        <?= htmlspecialchars($student["course"]) ?>
    </td>
</tr>
<?php endforeach; ?>
</table>
```

WHY htmlspecialchars()?

Database values may eventually be displayed inside HTML.

Using: htmlspecialchars() helps prevent HTML from being interpreted as markup.

Example: <?= htmlspecialchars(\$student["name"]) ?>

This should be used when displaying untrusted text in HTML contexts.

PDO + OOP ARCHITECTURE

Recommended structure:

```
student_system/
|
├── config/
|   └── Database.php
|
├── classes/
|   └── Student.php
|
├── create.php
├── index.php
├── edit.php
└── delete.php
```

DATABASE CLASS

```
config/Database.php
<?php

class Database
{
    private $host = "localhost";
    private $database = "student_system";
    private $username = "root";
    private $password = "";

    public function connect()
    {
        try {

            $pdo = new PDO(
                "mysql:host={$this->host};dbname={$this->database}",
                $this->username,
                $this->password
            );

            $pdo->setAttribute(
                PDO::ATTR_ERRMODE,
                PDO::ERRMODE_EXCEPTION
            );

            return $pdo;

        }
        catch(PDOException $e)
        {
            die("Database connection failed.");
        }
    }
}
```

STUDENT CLASS

```
classes/Student.php
<?php
class Student
{
    private $conn;

    public function __construct($conn)
    {
        $this->conn = $conn;
    }

    public function create(
```

```

        $student_id,
        $name,
        $email,
        $course
    )
    {
        $sql = "
            INSERT INTO students
            (student_id, name, email, course)
            VALUES
            (:student_id, :name, :email, :course)
        ";

        $stmt = $this->conn->prepare($sql);

        return $stmt->execute([
            "student_id" => $student_id,
            "name" => $name,
            "email" => $email,
            "course" => $course
        ]);
    }

    public function getAll()
    {
        $sql = "
            SELECT *
            FROM students
            ORDER BY id DESC
        ";

        $stmt = $this->conn->prepare($sql);

        $stmt->execute();

        return $stmt->fetchAll(PDO::FETCH_ASSOC);
    }

    public function getById($id)
    {
        $sql = "
            SELECT *
            FROM students
            WHERE id = :id
        ";

        $stmt = $this->conn->prepare($sql);

        $stmt->execute([

```

```

        "id" => $id
    ];

    return $stmt->fetch(PDO::FETCH_ASSOC);
}

public function update(
    $id,
    $student_id,
    $name,
    $email,
    $course
)
{
    $sql = "
        UPDATE students
        SET
            student_id = :student_id,
            name = :name,
            email = :email,
            course = :course
        WHERE id = :id
    ";

    $stmt = $this->conn->prepare($sql);

    return $stmt->execute([
        "id" => $id,
        "student_id" => $student_id,
        "name" => $name,
        "email" => $email,
        "course" => $course
    ]);
}

public function delete($id)
{
    $sql = "
        DELETE FROM students
        WHERE id = :id
    ";

    $stmt = $this->conn->prepare($sql);

    return $stmt->execute([
        "id" => $id
    ]);
}
}

```

USING THE STUDENT CLASS

Example:

```
require_once "config/Database.php";
require_once "classes/Student.php";

$databse = new Database();
$conn = $databse->connect();
$student = new Student($conn);
```

The object now has access to:

```
create()
getAll()
getById()
update()
delete()
```

CREATE OPERATION USING OOP + PDO

```
$student->create(
    "2026-004",
    "Ana Cruz",
    "ana@example.com",
    "BSIT"
);
```

READ OPERATION USING OOP + PDO

```
$students = $student->getAll();

foreach($students as $record)
{
    echo $record["name"];
}
```

READ ONE RECORD

```
$record = $student->getById(1);

echo $record["name"];
```

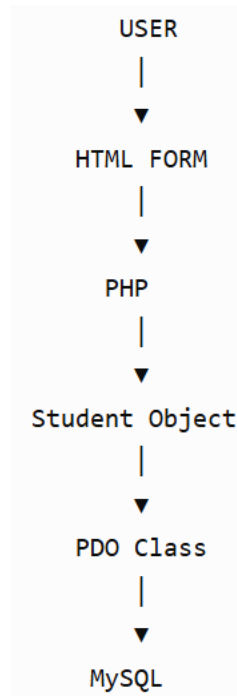
UPDATE USING OOP + PDO

```
$student->update(
    1,
    "2026-001",
    "Juan Updated",
    "juan@example.com",
    "BSCS"
);
```

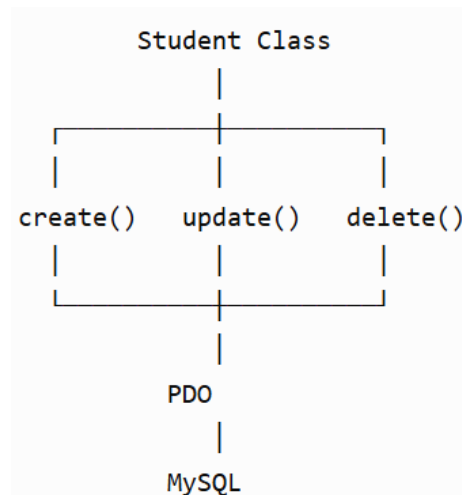
DELETE USING OOP + PDO

```
$student->delete(1);
```

The complete application now follows:



For CRUD:



PDO SECURITY

SQL Injection: SQL injection occurs when malicious SQL code is inserted into an application's database query.

Unsafe:

```

$id = $_GET["id"];
$sql = "SELECT * FROM students WHERE id = $id";
$result = $pdo->query($sql);
  
```

The problem is that user input becomes part of the SQL statement.

SECURE VERSION

```

$sql = "
  SELECT *
  
```

```
FROM students
WHERE id = :id
";
$stmt = $pdo->prepare($sql);

$stmt->execute([
    "id" => $id
]);
```

PDO SECURITY RULES

Rule 1 - Never directly concatenate user input into SQL.

Rule 2 - Use prepared statements.

Rule 3 - Validate input.

Rule 4 - Use appropriate data types.

Rule 5 - Escape output when displaying data in HTML.

Rule 6 - Do not expose database credentials unnecessarily.

Rule 7 - Handle database errors appropriately.